

# Spring Core - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

**Java Full Stack Master Course**

# Table of Contents

|                                                                                             |           |
|---------------------------------------------------------------------------------------------|-----------|
| <b>SPRING CORE — MASTER NOTES</b>                                                           | <b>6</b>  |
| A Complete Reference Book for Interviews, Production Work & System Understanding . . . . .  | 6         |
| <b>TOPIC 1: IOC (INVERSION OF CONTROL)</b>                                                  | <b>6</b>  |
| 1. Introduction . . . . .                                                                   | 6         |
| 1.1 What is it? . . . . .                                                                   | 6         |
| 1.2 Why was it introduced — The EXACT Problem It Solves . . . . .                           | 6         |
| 1.3 Interview Definition . . . . .                                                          | 7         |
| 1.4 IoC vs Dependency Injection — A Frequently Confused Pair . . . . .                      | 7         |
| 2. Real Life Analogy . . . . .                                                              | 7         |
| 3. Internal Working — The Spring IoC Container . . . . .                                    | 8         |
| 4. Real-World Testability Benefit — Made Concrete . . . . .                                 | 8         |
| 5. Common Mistakes . . . . .                                                                | 8         |
| 6. Best Practices . . . . .                                                                 | 9         |
| 7. Frequently Asked Interview Questions . . . . .                                           | 9         |
| <b>TOPIC 2: DEPENDENCY INJECTION</b>                                                        | <b>9</b>  |
| 1. Introduction . . . . .                                                                   | 9         |
| 2. The Three Types of DI — Full Code Comparison . . . . .                                   | 10        |
| Constructor Injection (STRONGLY RECOMMENDED — the modern default) . . . . .                 | 10        |
| Setter Injection (allows OPTIONAL dependencies, less common today) . . . . .                | 10        |
| Field Injection (convenient but DISCOURAGED in modern Spring) . . . . .                     | 10        |
| 3. Why Constructor Injection Is Strongly Preferred (Critical Interview Reasoning) . . . . . | 10        |
| 4. Real Life Analogy . . . . .                                                              | 11        |
| 5. Edge Cases . . . . .                                                                     | 11        |
| 6. Common Mistakes . . . . .                                                                | 11        |
| 7. Best Practices . . . . .                                                                 | 11        |
| 8. Frequently Asked Interview Questions . . . . .                                           | 11        |
| <b>TOPIC 3: BEANS</b>                                                                       | <b>12</b> |
| 1. Introduction . . . . .                                                                   | 12        |

|                                                                                                |    |
|------------------------------------------------------------------------------------------------|----|
| 2. Declaring a Bean — Two Main Ways . . . . .                                                  | 12 |
| Way 1: Stereotype Annotations (component scanning — covered fully in Topic 8) . . . . .        | 12 |
| Way 2: Explicit @Bean Method in a Configuration Class (covered fully in Topic 6) . . . . .     | 12 |
| 3. Real Life Analogy . . . . .                                                                 | 12 |
| 4. Retrieving a Bean Manually (Rarely Needed in Real Apps, but Useful to Understand) . . . . . | 13 |
| 5. Frequently Asked Interview Questions . . . . .                                              | 13 |

## **TOPIC 4: BEAN LIFECYCLE** **13**

|                                                            |    |
|------------------------------------------------------------|----|
| 1. Introduction . . . . .                                  | 13 |
| 2. The Full Lifecycle Diagram . . . . .                    | 14 |
| 3. Code Example — @PostConstruct and @PreDestroy . . . . . | 14 |
| 4. Real Life Analogy . . . . .                             | 14 |
| 5. Edge Cases . . . . .                                    | 15 |
| 6. Common Mistakes . . . . .                               | 15 |
| 7. Frequently Asked Interview Questions . . . . .          | 15 |

## **TOPIC 5: BEAN SCOPES** **15**

|                                                                    |    |
|--------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                          | 15 |
| 2. The Core Scopes . . . . .                                       | 16 |
| 3. Scope Comparison Table . . . . .                                | 16 |
| 4. Real Life Analogy . . . . .                                     | 16 |
| 5. Critical Interview Trap — Thread-Safety of Singletons . . . . . | 16 |
| 6. Edge Cases . . . . .                                            | 17 |
| 7. Common Mistakes . . . . .                                       | 17 |
| 8. Best Practices . . . . .                                        | 17 |
| 9. Frequently Asked Interview Questions . . . . .                  | 17 |

## **TOPIC 6: JAVA CONFIGURATION** **18**

|                                                                                        |    |
|----------------------------------------------------------------------------------------|----|
| 1. Introduction . . . . .                                                              | 18 |
| 2. Historical Context — Why This Replaced XML . . . . .                                | 18 |
| 3. Code Example — A Full @Configuration Class . . . . .                                | 18 |
| 4. When to Use @Bean (Java Config) vs @Component (Annotation-based, Topic 7) . . . . . | 18 |
| 5. Frequently Asked Interview Questions . . . . .                                      | 19 |

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>TOPIC 7: ANNOTATION CONFIGURATION</b>                                                     | <b>19</b> |
| 1. Introduction . . . . .                                                                    | 19        |
| 2. The Stereotype Annotations — All Functionally Identical, Semantically Different . . . . . | 19        |
| 3. Code Example — A Fully Annotation-Configured Set of Beans . . . . .                       | 20        |
| 4. Frequently Asked Interview Questions . . . . .                                            | 20        |
| <b>TOPIC 8: COMPONENT SCAN</b>                                                               | <b>21</b> |
| 1. Introduction . . . . .                                                                    | 21        |
| 2. Enabling Component Scanning . . . . .                                                     | 21        |
| 3. The Critical Rule — Package Structure Matters! . . . . .                                  | 21        |
| 4. Explicitly Widening the Scan (When Genuinely Needed) . . . . .                            | 21        |
| 5. Real Life Analogy . . . . .                                                               | 22        |
| 6. Edge Cases . . . . .                                                                      | 22        |
| 7. Common Mistakes . . . . .                                                                 | 22        |
| 8. Frequently Asked Interview Questions . . . . .                                            | 22        |
| <b>TOPIC 9: AUTOWIRING</b>                                                                   | <b>22</b> |
| 1. Introduction . . . . .                                                                    | 23        |
| 2. Autowiring by Type — The Default Behavior . . . . .                                       | 23        |
| 3. The Ambiguity Problem — Multiple Beans of the SAME Type . . . . .                         | 23        |
| 4. Resolving Ambiguity — @Qualifier and @Primary . . . . .                                   | 23        |
| 5. Autowiring a Collection of ALL Matching Beans . . . . .                                   | 24        |
| 6. Real Life Analogy . . . . .                                                               | 24        |
| 7. Edge Cases . . . . .                                                                      | 24        |
| 8. Common Mistakes . . . . .                                                                 | 24        |
| 9. Frequently Asked Interview Questions . . . . .                                            | 24        |
| <b>TOPIC 10: AOP BASICS (ASPECT-ORIENTED PROGRAMMING)</b>                                    | <b>25</b> |
| 1. Introduction . . . . .                                                                    | 25        |
| 2. Key AOP Terminology . . . . .                                                             | 25        |
| 3. Code Example — A Logging Aspect . . . . .                                                 | 26        |
| 4. The Five Advice Types . . . . .                                                           | 26        |
| 5. Real Life Analogy . . . . .                                                               | 26        |

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| 6. Real Production Usage — Where Spring Itself Uses AOP . . . . .                         | 27 |
| 7. Edge Cases . . . . .                                                                   | 27 |
| 8. Common Mistakes . . . . .                                                              | 27 |
| 9. Frequently Asked Interview Questions . . . . .                                         | 27 |
| 10. Revision Notes (Entire Spring Core Section Consolidated) . . . . .                    | 28 |
| 11. Homework (Covers Entire Spring Core Section) . . . . .                                | 28 |
| 12. Mini Project — A Fully Wired Notification System (Demonstrating Everything) . . . . . | 29 |
| FINAL SUMMARY — SPRING CORE COMPLETE . . . . .                                            | 29 |

# SPRING CORE — MASTER NOTES

*A Complete Reference Book for Interviews, Production Work & System Understanding*

---

## TOPIC 1: IOC (INVERSION OF CONTROL)

---

### 1. Introduction

#### 1.1 What is it?

IoC (Inversion of Control) is a design PRINCIPLE where the CONTROL of creating objects and wiring their dependencies together is TAKEN AWAY from your own application code and HANDED OVER to a framework/container — in Spring's case, the **Spring IoC Container**. Instead of your code saying "give me a new X," the container decides what to create, when, and hands it to you.

#### 1.2 Why was it introduced — The EXACT Problem It Solves

Let's look at code WITHOUT IoC — a typical Java class managing its own dependencies directly:

```
// THE "BEFORE SPRING" WORLD - the class creates its OWN dependencies directly
class OrderService {
    private PaymentProcessor paymentProcessor = new CreditCardPaymentProcessor(); // TIGHTLY COUPLED!
    private NotificationService notificationService = new EmailNotificationService(); // TIGHTLY COUPLED!

    void placeOrder(Order order) {
        paymentProcessor.charge(order.getAmount());
        notificationService.notify("Order placed!");
    }
}
```

This has SERIOUS problems:

1. **Tight coupling:** `OrderService` is PERMANENTLY welded to `CreditCardPaymentProcessor` and `EmailNotificationService`. Want to switch to PayPal, or SMS notifications? You must EDIT `OrderService`'s source code directly.
2. **Impossible to unit test in isolation:** You CANNOT test `OrderService`'s logic without ALSO triggering a real credit card charge and a real email send — there's no way to substitute a FAKE/mock payment processor for testing.
3. **No flexibility for different environments:** What if PRODUCTION needs `CreditCardPaymentProcessor` but TESTING needs a `FakePaymentProcessor`? With direct instantiation, you'd need different SOURCE CODE for each environment.

**With IoC**, the class instead just DECLARES what it NEEDS (an interface), and something EXTERNAL provides the actual implementation:

```
// THE "WITH SPRING/IoC" WORLD - dependencies are INJECTED from outside, not created internally
class OrderService {
    private final PaymentProcessor paymentProcessor; // just an INTERFACE reference
    private final NotificationService notificationService; // just an INTERFACE reference

    // Dependencies are PASSED IN (injected) from OUTSIDE, not created here
```

```

OrderService(PaymentProcessor paymentProcessor, NotificationService notificationService) {
    this.paymentProcessor = paymentProcessor;
    this.notificationService = notificationService;
}

void placeOrder(Order order) {
    paymentProcessor.charge(order.getAmount());
    notificationService.notify("Order placed!");
}
}

```

Now `OrderService` doesn't care WHICH specific `PaymentProcessor` it gets — production can wire in a real `CreditCardPaymentProcessor`, tests can wire in a `MockPaymentProcessor`, and NONE of `OrderService`'s own code ever changes. **The CONTROL over "which implementation gets used, and when it's created" has been INVERTED — moved from inside `OrderService` to an external coordinator (the Spring IoC Container).**

### 1.3 Interview Definition

"Inversion of Control is a design principle where the responsibility for creating objects and resolving their dependencies is transferred from the application's own code to an external framework/container, decoupling components from the concrete implementations they depend on and enabling flexible, testable, swappable designs."

### 1.4 IoC vs Dependency Injection — A Frequently Confused Pair

**IoC is the general PRINCIPLE** ("let something else control object creation/wiring"). **Dependency Injection (DI, next topic) is the SPECIFIC TECHNIQUE Spring uses to implement IoC** — passing dependencies IN (via constructor, setter, or field) rather than having a class look them up or create them itself. IoC is the "what/why"; DI is the specific "how."

## 2. Real Life Analogy

- **Restaurant:** WITHOUT IoC, imagine every single chef had to personally go grow their own vegetables, raise their own livestock, and manufacture their own kitchen equipment before they could cook anything — completely impractical and rigid. WITH IoC, a restaurant has a SEPARATE supply chain/procurement department that PROVIDES chefs with exactly the ingredients and equipment they need — the chef just DECLARES "I need fresh tomatoes," and the procurement system (the Spring Container) figures out and delivers the actual tomatoes, from whichever supplier is currently configured, without the chef needing to know or care where they came from.
- **Bank:** WITHOUT IoC, every bank TELLER personally manufactures their own calculator, forms, and stationery before starting work. WITH IoC, the bank's central OPERATIONS department provisions each teller with EXACTLY the standardized tools they need, and can swap out the SUPPLIER of those tools (calculators, forms) without the teller ever noticing or needing to change how they work.

### 3. Internal Working — The Spring IoC Container

```
SPRING IoC CONTAINER
(reads your configuration - annotations or XML/Java config)
|
v
+-----+
| 1. Discovers which classes should be managed ("BEANS") |
| 2. Creates instances of those classes |
| 3. Figures out each bean's DEPENDENCIES |
| 4. INJECTS those dependencies into each bean automatically |
| 5. Manages each bean's ENTIRE LIFECYCLE (creation -> use -> destroy)|
+-----+
|
v
YOUR APPLICATION CODE simply RECEIVES fully-wired,
ready-to-use objects - never manually instantiates them
```

The two main IoC Container implementations in Spring are `BeanFactory` (the basic, foundational container) and `ApplicationContext` (a more feature-rich container built on top of `BeanFactory`, adding things like event publishing, internationalization, and AOP support — `ApplicationContext` is what you'll use in virtually all real applications).

---

### 4. Real-World Testability Benefit — Made Concrete

```
// PRODUCTION code - Spring wires in the REAL payment processor automatically
@Service
class OrderService {
    private final PaymentProcessor paymentProcessor;

    @Autowired
    OrderService(PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }
}

// TEST code - a fake/mock is injected MANUALLY instead, with ZERO changes to OrderService itself!
@Test
void testPlaceOrder() {
    PaymentProcessor fakeProcessor = mock(PaymentProcessor.class); // a test double, no real charge happens
    OrderService service = new OrderService(fakeProcessor); // manually inject the FAKE dependency
    service.placeOrder(new Order(100.0));
    verify(fakeProcessor).charge(100.0); // verify the interaction, safely, in isolation
}
```

### 5. Common Mistakes

- Confusing IoC (the principle) with DI (the specific technique) as if they were entirely separate, unrelated concepts — DI is simply HOW Spring implements IoC.
- Continuing to manually instantiate dependencies with `new` inside Spring-managed classes, defeating the entire purpose of using the framework in the first place.



## 6. Best Practices

- Depend on INTERFACES/abstractions in your class fields, not concrete implementation classes — this is what makes IoC/DI genuinely powerful (swappable implementations).
- Let the Spring Container manage object creation/wiring entirely — avoid `new SomeSpringManagedClass()` inside your own application code.

## 7. Frequently Asked Interview Questions

- 1 **What is Inversion of Control?** A design principle where object creation and dependency wiring is handed over to an external framework/container, rather than being handled by the class itself.
  - 2 **What specific problem does IoC solve?** Tight coupling between a class and its concrete dependencies, which makes code hard to test in isolation, hard to swap implementations, and hard to configure differently across environments.
  - 3 **What's the difference between IoC and Dependency Injection?** IoC is the general PRINCIPLE; Dependency Injection is the SPECIFIC TECHNIQUE (passing dependencies in via constructor/setter/field) that Spring uses to implement that principle.
  - 4 **What are the two main Spring IoC Container implementations?** `BeanFactory` (basic/foundational) and `ApplicationContext` (feature-rich, used in virtually all real applications).
  - 5 **How does IoC improve testability?** Because dependencies are passed IN rather than created internally, tests can substitute FAKE/mock implementations without modifying the class under test at all.
- 
- 

# TOPIC 2: DEPENDENCY INJECTION

---

## 1. Introduction

**What is it?** Dependency Injection (DI) is the specific TECHNIQUE Spring uses to implement IoC (Topic 1) — a dependency is "injected" (passed in) to a class from OUTSIDE, rather than the class creating it or looking it up itself, via one of three mechanisms: constructor injection, setter injection, or field injection.

## 2. The Three Types of DI — Full Code Comparison

### Constructor Injection (**STRONGLY RECOMMENDED** — the modern default)

```
@Service
public class OrderService {
    private final PaymentProcessor paymentProcessor; // can be FINAL - immutable after construction!

    @Autowired // optional on a single constructor since Spring 4.3+, but good to be explicit
    public OrderService(PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }
}
```

### Setter Injection (allows **OPTIONAL** dependencies, less common today)

```
@Service
public class OrderService {
    private PaymentProcessor paymentProcessor; // CANNOT be final - must be mutable for the setter

    @Autowired
    public void setPaymentProcessor(PaymentProcessor paymentProcessor) {
        this.paymentProcessor = paymentProcessor;
    }
}
```

### Field Injection (convenient but **DISCOURAGED** in modern Spring)

```
@Service
public class OrderService {
    @Autowired
    private PaymentProcessor paymentProcessor; // injected DIRECTLY into the field via reflection
}
```

## 3. Why Constructor Injection Is Strongly Preferred (Critical Interview Reasoning)

| Reason                                                          | Explanation                                                                                                                                              |
|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Immutability</b>                                             | Constructor injection allows the field to be <code>final</code> — guaranteed to never change after construction, a safer design                          |
| <b>Guaranteed complete initialization</b>                       | An object literally CANNOT be constructed without its required dependencies — impossible to end up in a "half-wired" state                               |
| <b>Testability WITHOUT Spring</b>                               | You can create the object with <code>new OrderService(fakeProcessor)</code> directly in a plain unit test, with ZERO Spring framework involvement at all |
| <b>Makes dependencies explicit and visible</b>                  | Looking at the constructor signature IMMEDIATELY tells you everything this class needs — field injection HIDES dependencies inside the class body        |
| <b>Prevents circular dependency issues from going unnoticed</b> | Constructor injection FAILS FAST (at startup) if a circular dependency exists; field injection can sometimes mask this problem longer                    |

```
// Field injection - testing requires REFLECTION tricks or a full Spring context just to set the field:
OrderService service = new OrderService();
// paymentProcessor is null! You'd need Spring's test context OR reflection to inject it manually.
```

```
// Constructor injection - testing is TRIVIAL, completely Spring-free:  
OrderService service = new OrderService(fakeProcessor); // just... works, directly
```

## 4. Real Life Analogy

- **Bank:** Constructor injection is like a new bank teller being handed ALL their required tools (cash drawer, ID badge, terminal login) AT THE MOMENT they're hired — they simply CANNOT start working in an incomplete, half-equipped state. Field injection is like handing the teller their badge SEPARATELY, sometime AFTER they've already started sitting at their desk — there's an awkward window where they exist but aren't fully equipped yet, which is exactly the kind of incomplete state constructor injection eliminates entirely.

## 5. Edge Cases

| Case                                                                           | Result                                                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A class has MULTIPLE constructors, none marked <code>@Autowired</code>         | Spring requires an EXPLICIT no-arg constructor or throws an error trying to decide which to use                                                                                                                                                                                                       |
| A REQUIRED dependency isn't available in the Spring context at all             | <code>NoSuchBeanDefinitionException</code> at application STARTUP (fails fast) — a GOOD thing, catching the misconfiguration immediately rather than at some unpredictable runtime moment                                                                                                             |
| Two beans depend on EACH OTHER via constructor injection (circular dependency) | Spring throws a <code>BeanCurrentlyInCreationException</code> at startup — constructor injection cannot resolve genuine circular dependencies (setter/field injection technically CAN, via a multi-pass workaround, but a circular dependency is usually itself a DESIGN SMELL worth fixing directly) |

## 6. Common Mistakes

- Defaulting to field injection out of habit/convenience (it "looks" simpler) — modern Spring guidance strongly favors constructor injection instead.
- Designing genuinely circular dependencies between beans — usually a sign that responsibilities need to be reorganized/extracted into a third class, not a problem to "work around" with setter injection.

## 7. Best Practices

- Use constructor injection as the DEFAULT choice for REQUIRED dependencies.
- Reserve setter injection for genuinely OPTIONAL dependencies (rare in practice).
- Avoid field injection in production code — it hides dependencies, prevents `final` fields, and makes plain-Java unit testing (without Spring) much harder.

## 8. Frequently Asked Interview Questions

- 1 **What are the three types of Dependency Injection in Spring?** Constructor injection, setter injection, field injection.
- 2 **Which type is recommended, and why?** Constructor injection — it enables `final`/immutable fields, guarantees the object is never in a half-initialized state, and is trivially testable without any Spring framework involvement at all.
- 3 **Why is field injection discouraged?** It hides a class's actual dependencies (not visible in the constructor signature), prevents `final` fields, and makes plain unit testing harder (typically requiring reflection or a Spring test context just to inject a value).
- 4 **What happens if a required dependency can't be found in the Spring context?** Spring throws `NoSuchBeanDefinitionException` at application STARTUP — a deliberate "fail fast" behavior, catching configuration errors immediately.
- 5 **Can constructor injection handle circular dependencies between two beans?** No — it fails at startup with `BeanCurrentlyInCreationException`; this is usually a sign of a genuine design problem needing to be fixed, not something to "work around."

---

## TOPIC 3: BEANS

---

### 1. Introduction

**What is it?** A "bean" is simply Java's term (specifically, Spring's term) for any OBJECT that is CREATED, CONFIGURED, and MANAGED by the Spring IoC Container — as opposed to an ordinary object you create yourself with `new`.

### 2. Declaring a Bean — Two Main Ways

**Way 1: Stereotype Annotations (component scanning — covered fully in Topic 8)**

```
@Service // marks this class as a Spring-managed bean (specifically, a "service" layer bean)
public class OrderService {
    // ...
}
```

**Way 2: Explicit `@Bean` Method in a Configuration Class (covered fully in Topic 6)**

```
@Configuration
public class AppConfig {
    @Bean
    public PaymentProcessor paymentProcessor() {
        return new CreditCardPaymentProcessor(); // Spring calls THIS method and manages the RETURNED object
    }
}
```

### 3. Real Life Analogy

- **Restaurant:** A "bean" is like any piece of KITCHEN EQUIPMENT that the restaurant's central procurement system (Spring Container) has officially registered, tracked, and is responsible for maintaining/replacing —

as opposed to a chef's PERSONAL utensil they brought from home that the restaurant's system knows nothing about and isn't responsible for.

## 4. Retrieving a Bean Manually (Rarely Needed in Real Apps, but Useful to Understand)

```
ApplicationContext context = SpringApplication.run(MyApp.class, args);
OrderService orderService = context.getBean(OrderService.class);
```

In real Spring Boot applications, you almost NEVER call `getBean()` yourself — instead, you let Spring AUTOMATICALLY inject beans into OTHER beans via `@Autowired` (covered in Topic 9). Manually retrieving beans this way is mostly useful for understanding what's happening, or in rare bootstrapping scenarios.

## 5. Frequently Asked Interview Questions

- 1 **What is a "bean" in Spring?** Any object whose creation and lifecycle is managed by the Spring IoC Container, as opposed to an object you instantiate yourself with `new`.
- 2 **Name two ways to declare a bean.** Stereotype annotations (`@Component`/`@Service`/`@Repository`/`@Controller`), or an explicit `@Bean`-annotated method inside a `@Configuration` class.
- 3 **In a real application, how do you typically get access to a bean?** By having Spring AUTOMATICALLY inject it into another bean (via `@Autowired`), not by manually calling `context.getBean()`.

---

# TOPIC 4: BEAN LIFECYCLE

---

## 1. Introduction

**What is it?** Every Spring bean goes through a well-defined, predictable sequence of phases from creation to destruction — understanding this lifecycle lets you HOOK IN custom initialization/cleanup logic at exactly the right moment.

## 2. The Full Lifecycle Diagram

```
1. INSTANTIATION -> Spring creates the object (calls its constructor)
2. POPULATE PROPERTIES -> Spring injects dependencies (constructor/setter/field DI)
3. BeanNameAware, BeanFactoryAware, ApplicationContextAware (if implemented) -> Spring
calls these "Aware" interfaces' setter methods, if the bean
needs to know its own name or access the container itself
4. @PostConstruct -> YOUR custom initialization logic runs (e.g., "load a cache")
5. InitializingBean.afterPropertiesSet() -> an alternative, interface-based init hook
|
v
[ BEAN IS NOW FULLY READY AND IN USE - handling application requests/logic ]
|
v (application shutdown)
6. @PreDestroy -> YOUR custom cleanup logic runs (e.g., "close a connection")
7. DisposableBean.destroy() -> an alternative, interface-based cleanup hook
|
v
[ BEAN IS DESTROYED / GARBAGE COLLECTED ]
```

## 3. Code Example — @PostConstruct and @PreDestroy

```
import jakarta.annotation.PostConstruct;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Component;

@Component
public class CacheWarmupService {

    @PostConstruct
    public void warmUpCache() {
        System.out.println("Bean fully constructed and wired - warming up cache now...");
        // load frequently-used data into an in-memory cache, for example
    }

    @PreDestroy
    public void releaseResources() {
        System.out.println("Application shutting down - releasing cache resources...");
    }
}
```

## 4. Real Life Analogy

- **Restaurant:** The bean lifecycle is like a new chef's onboarding process — hired (instantiated), given their equipment/ingredients (dependencies injected), THEN they do their own personal station setup (@PostConstruct — sharpening knives, prepping their station) BEFORE actually cooking for customers, and finally packing away/cleaning their station (@PreDestroy) at the end of their shift, before leaving entirely.

## 5. Edge Cases

| Case                                                                                                           | Result                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| An exception thrown inside <code>@PostConstruct</code>                                                         | Typically PREVENTS the application context from starting up successfully — a fail-fast safety mechanism                                                                                   |
| Relying on another bean's state inside <code>@PostConstruct</code> before that OTHER bean is fully initialized | Can cause subtle bugs — dependency ORDER matters; Spring generally initializes dependencies BEFORE the beans that need them, but complex multi-bean initialization ordering deserves care |

## 6. Common Mistakes

- Putting expensive, slow logic in `@PostConstruct` without realizing it DELAYS the entire application's startup time.
- Forgetting `@PreDestroy` cleanup logic for beans holding external resources (file handles, network connections), causing resource leaks on shutdown.

## 7. Frequently Asked Interview Questions

- 1 **What are the key phases of a Spring bean's lifecycle?** Instantiation, dependency injection, `@PostConstruct` (custom init logic), active use, `@PreDestroy` (custom cleanup logic), destruction.
- 2 **What does `@PostConstruct` do?** Marks a method to run automatically AFTER the bean is fully constructed and all its dependencies are injected — the standard place for custom initialization logic.
- 3 **What does `@PreDestroy` do?** Marks a method to run automatically just BEFORE the bean is destroyed (e.g., on application shutdown) — the standard place for cleanup logic.
- 4 **What happens if `@PostConstruct` throws an exception?** It typically prevents the Spring application context from starting up successfully at all.

---

# TOPIC 5: BEAN SCOPES

---

## 1. Introduction

**What is it?** A bean's SCOPE determines HOW MANY INSTANCES of it Spring creates, and for HOW LONG each instance lives — by default, Spring beans are `singleton` (exactly ONE shared instance for the entire application), but other scopes exist for different needs.

## 2. The Core Scopes

```
@Service
@Scope("singleton") // DEFAULT - ONE shared instance for the ENTIRE application
public class ConfigService { }

@Component
@Scope("prototype") // a BRAND-NEW instance is created EVERY time it's requested/injected
public class ReportGenerator { }

@Component
@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS) // ONE instance PER HTTP request
(web apps only)
public class RequestAuditLogger { }

@Component
@Scope(value = "session", proxyMode = ScopedProxyMode.TARGET_CLASS) // ONE instance PER USER SESSION
(web apps only)
public class ShoppingCart { }
```

## 3. Scope Comparison Table

| Scope               | Instances Created                               | Typical Use Case                                         |
|---------------------|-------------------------------------------------|----------------------------------------------------------|
| singleton (DEFAULT) | Exactly ONE, shared everywhere                  | Stateless services, config, most beans                   |
| prototype           | A NEW instance every single time it's requested | Genuinely STATEFUL, non-shareable objects (rare)         |
| request             | ONE per HTTP request (web apps)                 | Per-request state (e.g., a request-specific audit trail) |
| session             | ONE per user HTTP session (web apps)            | Per-user state (e.g., a shopping cart)                   |

## 4. Real Life Analogy

- **Bank:** A `singleton` bean is like the bank's SINGLE central exchange-rate calculator — everyone shares and consults the EXACT SAME instance. A `prototype` bean is like a fresh, individually-issued deposit slip generated FRESH every single time a customer needs one — never reused or shared between customers.

## 5. Critical Interview Trap — Thread-Safety of Singletons

Because `singleton` is the OVERWHELMING default, and ONE shared instance handles MANY CONCURRENT requests (exactly the same concern as a Servlet instance — see Advanced Java Topic 1), **singleton beans MUST be stateless (or properly thread-safe) if they'll be accessed concurrently** — mutable instance fields on a singleton bean create the EXACT same race-condition risk covered in Core Java's Synchronization topic.

```
@Service
public class BadCounterService {
    private int requestCount = 0; // DANGEROUS! Shared, mutable state on a SINGLETON bean

    public void handleRequest() {
        requestCount++; // RACE CONDITION across concurrent requests!
    }
}
```



## 6. Edge Cases

| Case                                                                                                                          | Result                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Injecting a <code>prototype</code> -scoped bean into a <code>singleton</code> -scoped bean NAIVELY (a normal field injection) | The <code>prototype</code> bean is created ONLY ONCE (at the singleton's own creation time) and then REUSED forever after — a common, surprising gotcha; a special "scoped proxy" or <code>ObjectProvider/ObjectFactory</code> pattern is needed to get a genuinely FRESH prototype instance on each actual use |
| Using <code>request/session</code> scope OUTSIDE a genuine web application context                                            | Error — these scopes only make sense within an active HTTP request/session                                                                                                                                                                                                                                      |

## 7. Common Mistakes

- Storing mutable, request-specific state directly as instance fields on a `singleton` bean — a serious concurrency bug, exactly like the Servlet instance-field mistake from Advanced Java.
- Expecting a `prototype` bean injected into a `singleton` to be freshly created on every USE — it's only created once, at the singleton's OWN construction time, unless special scoped-proxy handling is used.

## 8. Best Practices

- Keep `singleton` beans (the vast majority of your beans) STATELESS — use local variables for anything request/call-specific, never mutable instance fields.
- Reserve `prototype/request/session` scopes for the genuinely rare cases where per-instance/per-request/per-session state is truly needed.

## 9. Frequently Asked Interview Questions

- 1 **What is the default Spring bean scope?** `singleton` — exactly one shared instance for the entire application.
  - 2 **What's the difference between `singleton` and `prototype` scope?** `singleton` creates ONE shared instance reused everywhere; `prototype` creates a BRAND-NEW instance every time the bean is requested/injected.
  - 3 **Why must `singleton` beans generally be stateless?** Because ONE shared instance handles MANY CONCURRENT requests/threads — mutable instance fields create race conditions, exactly like a shared Servlet instance field.
  - 4 **What's a common gotcha when injecting a `prototype` bean into a `singleton` bean?** The prototype is only created ONCE (at the singleton's own construction), not freshly on every actual use, unless a scoped proxy or `ObjectProvider` pattern is used instead.
- 
-

# TOPIC 6: JAVA CONFIGURATION

## 1. Introduction

**What is it?** Java Configuration is a way of defining Spring beans EXPLICITLY, in plain Java code, using `@Configuration` classes with `@Bean`-annotated methods — a fully type-safe, refactorable alternative to the OLDER, now largely obsolete, XML-based configuration approach.

## 2. Historical Context — Why This Replaced XML

Early Spring (2000s) configured EVERYTHING via large, verbose XML files:

```
<!-- OLD, XML-based Spring configuration (largely obsolete today) -->
<bean id="paymentProcessor" class="com.company.CreditCardPaymentProcessor" />
<bean id="orderService" class="com.company.OrderService">
  <constructor-arg ref="paymentProcessor" />
</bean>
```

This was verbose, NOT type-safe (typos in class names only caught at RUNTIME), and disconnected from the actual Java code it configured. Java Configuration (introduced in Spring 3.0, 2009) fixed all of this.

## 3. Code Example — A Full `@Configuration` Class

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration // marks this class as a source of bean definitions
public class AppConfig {

    @Bean // Spring calls this method ONCE, manages the returned object as a bean
    public PaymentProcessor paymentProcessor() {
        return new CreditCardPaymentProcessor();
    }

    @Bean
    public OrderService orderService(PaymentProcessor paymentProcessor) { // Spring INJECTS the other
        @Bean here
        return new OrderService(paymentProcessor);
    }
}
```

## 4. When to Use `@Bean` (Java Config) vs `@Component` (Annotation-based, Topic 7)

| Scenario                                                                                               | Recommendation                                                                                                                                    |
|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| You WROTE the class yourself, and it's part of your own codebase                                       | Use <code>@Component</code> / <code>@Service</code> /etc. stereotype annotations directly on the class (simpler, less boilerplate)                |
| The class comes from a THIRD-PARTY LIBRARY (you can't add annotations to it)                           | MUST use a <code>@Bean</code> method in a <code>@Configuration</code> class — this is the ONLY way to make Spring manage a class you can't modify |
| The object needs COMPLEX construction logic (conditional setup, multiple constructor argument choices) | A <code>@Bean</code> method gives you FULL Java code flexibility to construct it exactly as needed                                                |

## 5. Frequently Asked Interview Questions

- 1 **What is Java Configuration in Spring?** Defining beans explicitly via `@Configuration` classes with `@Bean`-annotated methods, in plain, type-safe Java code.
- 2 **What did Java Configuration replace?** Verbose, non-type-safe XML-based Spring configuration.
- 3 **When would you use `@Bean` instead of a stereotype annotation like `@Component`?** When the class comes from a third-party library you cannot add annotations to, or when its construction requires complex custom logic.

---

# TOPIC 7: ANNOTATION CONFIGURATION

---

## 1. Introduction

**What is it?** Annotation Configuration refers to using STEREO TYPE ANNOTATIONS (`@Component`, `@Service`, `@Repository`, `@Controller`) placed DIRECTLY on your own classes, letting Spring automatically discover and register them as beans via component scanning (full detail in Topic 8) — the most common, convenient way to declare beans for code you own.

## 2. The Stereotype Annotations — All Functionally Identical, Semantically Different

```
@Component // the GENERIC, base stereotype - "this is a Spring-managed bean"
public class GenericHelper { }

@Service // semantically: "this is a BUSINESS LOGIC / service layer bean"
public class OrderService { }

@Repository // semantically: "this is a DATA ACCESS layer bean" - ALSO enables automatic
public class EmployeeRepository { } // translation of database exceptions into Spring's unified
exception hierarchy

@Controller // semantically: "this is a WEB LAYER bean handling HTTP requests" (classic MVC, returns
views)
public class HomeController { }

@RestController // = @Controller + @ResponseBody combined - returns data (JSON) directly, not a view
name
public class ApiController { }
```

**Key insight:** `@Service`, `@Repository`, and `@Controller` are all internally just SPECIALIZED versions of `@Component` (they're annotated WITH `@Component` themselves) — functionally, Spring treats them almost identically for bean registration purposes. Using the SPECIFIC one (rather than always `@Component`) is primarily about clearly COMMUNICATING each class's architectural role (matching the MVC/layered architecture from Advanced Java Topic 2), plus `@Repository` specifically adds automatic exception translation.

### 3. Code Example — A Fully Annotation-Configured Set of Beans

```
@Repository
public class EmployeeRepository {
    public Employee findById(Long id) { /* ... database access logic ... */ return null; }
}

@Service
public class EmployeeService {
    private final EmployeeRepository repository;

    @Autowired
    public EmployeeService(EmployeeRepository repository) { // Spring automatically finds and injects the
        @Repository bean
        this.repository = repository;
    }

    public Employee getEmployee(Long id) {
        return repository.findById(id);
    }
}

@RestController
public class EmployeeController {
    private final EmployeeService service;

    @Autowired
    public EmployeeController(EmployeeService service) { // Spring automatically finds and injects the
        @Service bean
        this.service = service;
    }

    @GetMapping("/employees/{id}")
    public Employee getEmployee(@PathVariable Long id) {
        return service.getEmployee(id);
    }
}
```

Notice the ENTIRE application (Controller → Service → Repository) is wired together AUTOMATICALLY — you never write a single `new EmployeeService(...)` anywhere; Spring discovers all three classes, creates them, and wires them together based purely on these annotations.

### 4. Frequently Asked Interview Questions

- 1 **What's the difference between `@Component`, `@Service`, `@Repository`, and `@Controller`?** They're functionally near-identical (`@Service/@Repository/@Controller` are all specialized forms of `@Component`), primarily differing in SEMANTIC meaning to communicate architectural layer; `@Repository` additionally enables automatic database exception translation.
- 2 **What does `@RestController` combine?** `@Controller` + `@ResponseBody` — meaning methods return data directly (typically serialized to JSON) instead of a view name to render.
- 3 **Why use `@Service` instead of just always using `@Component` everywhere?** For CLARITY — it immediately communicates the class's architectural role to other developers reading the code, aligning with the layered MVC architecture.

# TOPIC 8: COMPONENT SCAN

## 1. Introduction

**What is it?** Component scanning is the mechanism by which Spring AUTOMATICALLY searches through specified packages, finds classes annotated with stereotype annotations (`@Component` and its specializations), and registers them as beans — WITHOUT you needing to manually declare each one via a `@Bean` method.

## 2. Enabling Component Scanning

```
@SpringBootApplication // in Spring Boot, this ALREADY includes @ComponentScan automatically!
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

`@SpringBootApplication` is actually a CONVENIENCE annotation combining THREE separate annotations:

```
@SpringBootConfiguration // = @Configuration (this class can define @Bean methods too)
@EnableAutoConfiguration // Spring Boot's "magic" auto-configuration (covered in the Spring Boot
section)
@ComponentScan // scans the CURRENT package and all its SUB-packages for @Component classes
public @interface SpringBootApplication { }
```

## 3. The Critical Rule — Package Structure Matters!

```
com.company.myapp
■■■ MyApplication.java <- @SpringBootApplication HERE
■■■ controller
■ ■■■ EmployeeController.java <- FOUND (sub-package of com.company.myapp)
■■■ service
■ ■■■ EmployeeService.java <- FOUND (sub-package of com.company.myapp)
■■■ repository
■■■ EmployeeRepository.java <- FOUND (sub-package of com.company.myapp)

com.othercompany.utils
■■■ SomeHelper.java <- NOT FOUND! Different top-level package entirely,
OUTSIDE the scanned tree - Spring has NO IDEA it exists
```

**Critical, frequently-tested gotcha:** `@ComponentScan` (via `@SpringBootApplication`) only scans the package containing the main application class, AND ITS SUB-PACKAGES. A `@Component` class placed in a completely SEPARATE, unrelated package tree will simply be INVISIBLE to Spring, silently never registered as a bean — causing a confusing `NoSuchBeanDefinitionException` elsewhere when something tries to inject it.

## 4. Explicitly Widening the Scan (When Genuinely Needed)

```
@SpringBootApplication
@ComponentScan(basePackages = {"com.company.myapp", "com.othercompany.utils"}) // explicitly ADD
another package tree
public class MyApplication { }
```

## 5. Real Life Analogy

- **College:** Component scanning is like a college automatically registering every STUDENT located WITHIN its own campus buildings (the main package and sub-packages) — a student who happens to be at an ENTIRELY DIFFERENT, unaffiliated campus (a different top-level package) simply never gets discovered or registered, unless the college explicitly extends its enrollment search to include that other campus too.

## 6. Edge Cases

| Case                                                                                       | Result                                                                                                                                                              |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A <code>@Component</code> class in a package OUTSIDE the scanned tree                      | Never registered as a bean — silently invisible to Spring, no error at THAT point, but a confusing failure later when something tries to <code>@Autowired</code> it |
| The main <code>@SpringBootApplication</code> class moved to a DEEPER package than intended | Can accidentally NARROW the scan, missing sibling packages that used to be included                                                                                 |

## 7. Common Mistakes

- Placing the main `@SpringBootApplication` class in the WRONG package (too deep, or in a package with no relation to the rest of the codebase), accidentally narrowing what gets scanned.
- Being confused by `NoSuchBeanDefinitionException` without realizing the actual root cause is that the bean's class simply lives OUTSIDE the scanned package tree.

## 8. Frequently Asked Interview Questions

- 1 **What is component scanning?** Spring automatically searching specified packages (and sub-packages) for stereotype-annotated classes and registering them as beans.
- 2 **What's the default scanning scope with `@SpringBootApplication`?** The package containing the main application class, and ALL its sub-packages.
- 3 **What happens to a `@Component` class placed in a package OUTSIDE that scanned tree?** It's simply never discovered/registered as a bean, silently — often surfacing later as a confusing `NoSuchBeanDefinitionException` when something else tries to inject it.
- 4 **How do you widen component scanning to include additional packages?** Use `@ComponentScan(basePackages = {...})` explicitly listing the additional package(s).

---

# TOPIC 9: AUTOWIRING

---

## 1. Introduction

**What is it?** Autowiring is Spring's mechanism for AUTOMATICALLY resolving and injecting a bean's dependencies — you mark WHERE a dependency is needed (`@Autowired`), and Spring figures out WHICH bean to provide, based on TYPE (and, when needed, NAME/qualifier) matching.

## 2. Autowiring by Type — The Default Behavior

```
public interface PaymentProcessor { void charge(double amount); }

@Component
public class CreditCardPaymentProcessor implements PaymentProcessor { ... }

@Service
public class OrderService {
    @Autowired
    public OrderService(PaymentProcessor paymentProcessor) { // Spring finds the ONE bean implementing
        this TYPE
    }
    // ...
}
```

## 3. The Ambiguity Problem — Multiple Beans of the SAME Type

```
@Component
public class CreditCardPaymentProcessor implements PaymentProcessor { }

@Component
public class PayPalPaymentProcessor implements PaymentProcessor { }

@Service
public class OrderService {
    @Autowired
    public OrderService(PaymentProcessor paymentProcessor) { } // AMBIGUOUS! Which one? Spring doesn't
    know!
}
// Startup FAILS with: NoUniqueBeanDefinitionException
```

## 4. Resolving Ambiguity — `@Qualifier` and `@Primary`

```
// FIX 1: @Qualifier - explicitly specify WHICH bean by NAME
@Component("creditCardProcessor")
public class CreditCardPaymentProcessor implements PaymentProcessor { }

@Service
public class OrderService {
    @Autowired
    public OrderService(@Qualifier("creditCardProcessor") PaymentProcessor paymentProcessor) { }
}

// FIX 2: @Primary - mark ONE implementation as the DEFAULT choice when ambiguous
@Component
@Primary // this one wins UNLESS a @Qualifier says otherwise
public class CreditCardPaymentProcessor implements PaymentProcessor { }

@Component
public class PayPalPaymentProcessor implements PaymentProcessor { }
```

## 5. Autowiring a Collection of ALL Matching Beans

```
@Service
public class NotificationDispatcher {
    private final List<NotificationService> allNotificationServices;

    @Autowired
    public NotificationDispatcher(List<NotificationService> allNotificationServices) {
        this.allNotificationServices = allNotificationServices; // Spring injects EVERY bean implementing this
        interface!
    }

    public void notifyAll(String message) {
        allNotificationServices.forEach(service -> service.notify(message));
    }
}
```

## 6. Real Life Analogy

- **Restaurant:** Autowiring by type is like a chef asking for "a knife" and the kitchen procurement system automatically providing the ONE knife available. If there are MULTIPLE knife types (paring, chef's, bread), the system needs EITHER a specific label (`@Qualifier` — "give me specifically the bread knife") OR a designated DEFAULT knife (`@Primary` — "unless specified otherwise, give the chef's knife").

## 7. Edge Cases

| Case                                                                                 | Result                                                                    |
|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Multiple beans of the SAME type exist, no <code>@Qualifier/@Primary</code> specified | <code>NoUniqueBeanDefinitionException</code> at startup                   |
| NO bean of the required type exists at all                                           | <code>NoSuchBeanDefinitionException</code> at startup                     |
| <code>@Autowired(required = false)</code> on a dependency that doesn't exist         | Injects null instead of throwing — allows genuinely OPTIONAL dependencies |

## 8. Common Mistakes

- Adding a second implementation of an interface without realizing it breaks existing autowiring elsewhere with a sudden `NoUniqueBeanDefinitionException`.
- Overusing `@Primary` everywhere as a blanket "fix" for ambiguity, instead of being deliberate about WHICH specific implementation each injection point actually needs (`@Qualifier` is often the more precise, intentional choice).

## 9. Frequently Asked Interview Questions

- 1 **How does Spring resolve which bean to inject by default?** By matching the required TYPE — if exactly one bean implements/matches that type, it's injected automatically.
- 2 **What happens if multiple beans match the same type?** `NoUniqueBeanDefinitionException` at startup, UNLESS resolved via `@Qualifier` (explicitly naming which bean) or `@Primary` (marking a default).



- 3 **What's the difference between `@Qualifier` and `@Primary`?** `@Qualifier` explicitly specifies exactly which bean by name at each injection point; `@Primary` marks ONE bean as the default choice used whenever ambiguity exists and no `@Qualifier` overrides it.
  - 4 **Can you autowire a List of all beans implementing an interface?** Yes — Spring will inject a list containing EVERY matching bean automatically.
- 

## TOPIC 10: AOP BASICS (ASPECT-ORIENTED PROGRAMMING)

---

### 1. Introduction

**What is it?** AOP lets you define CROSS-CUTTING CONCERNS (logging, security checks, transaction management) SEPARATELY from your core business logic, then have Spring AUTOMATICALLY "weave" that concern into MULTIPLE methods/classes — WITHOUT manually duplicating that logic (like a logging statement) inside every single method that needs it.

**Why introduced:** Concerns like logging or transaction management genuinely apply ACROSS many, otherwise-unrelated classes/methods — hand-writing the SAME boilerplate (`System.out.println("Method X called")`) inside every single business method is repetitive AND clutters the actual business logic. AOP lets you define this ONCE, in ONE place, and apply it EVERYWHERE it's needed, automatically — this is conceptually the SAME idea as a Servlet Filter (Advanced Java Topic 6) intercepting requests, just applied at the METHOD level instead of the HTTP request level.

### 2. Key AOP Terminology

| Term       | Meaning                                                                                    |
|------------|--------------------------------------------------------------------------------------------|
| Aspect     | A MODULE containing cross-cutting logic (e.g., a <code>LoggingAspect</code> class)         |
| Advice     | The ACTUAL ACTION taken (e.g., "log before the method runs")                               |
| Join Point | A POINT during execution where advice CAN be applied (in Spring AOP, always a method call) |
| Pointcut   | An EXPRESSION defining WHICH join points (which methods) the advice actually applies to    |
| Weaving    | The PROCESS of actually linking aspects into the target code at runtime                    |

### 3. Code Example — A Logging Aspect

```
import org.aspectj.lang.annotation.*;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before("execution(* com.company.service.*(..))") // POINTCUT: any method in any class in this
    package
    public void logBefore(org.aspectj.lang.JoinPoint joinPoint) {
        System.out.println("Calling method: " + joinPoint.getSignature().getName());
    }

    @AfterReturning(pointcut = "execution(* com.company.service.*(..))", returning = "result")
    public void logAfter(org.aspectj.lang.JoinPoint joinPoint, Object result) {
        System.out.println("Method " + joinPoint.getSignature().getName() + " returned: " + result);
    }

    @Around("execution(* com.company.service.OrderService.placeOrder(..))")
    public Object measureExecutionTime(org.aspectj.lang.ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = joinPoint.proceed(); // actually RUNS the intercepted method
        long duration = System.currentTimeMillis() - start;
        System.out.println("Execution took: " + duration + "ms");
        return result;
    }
}
```

**NOTICE:** `OrderService` (and every other class in `com.company.service`) has ZERO knowledge of this logging aspect — it never calls it, never imports it. Spring AUTOMATICALLY "weaves" this cross-cutting logic in at runtime, based purely on the pointcut expression, applying to EVERY matching method without a single line of that class's own code being touched.

### 4. The Five Advice Types

| Advice          | Runs                                                                                                                    |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| @Before         | BEFORE the target method executes                                                                                       |
| @After          | AFTER the target method completes (regardless of success/exception)                                                     |
| @AfterReturning | AFTER the target method completes SUCCESSFULLY (has access to the return value)                                         |
| @AfterThrowing  | AFTER the target method throws an EXCEPTION                                                                             |
| @Around         | WRAPS the ENTIRE method call — can run logic both BEFORE and AFTER, and can even PREVENT the method from running at all |

### 5. Real Life Analogy

- **Airport security:** AOP is exactly like the airport security checkpoint from the Filters analogy (Advanced Java Topic 6) — but applied at the METHOD level instead of the HTTP-request level: EVERY passenger (method call) matching a certain criteria (the pointcut) automatically passes through the SAME standardized check (the advice), without each individual gate (business method) needing its own separate security logic written into it.

## 6. Real Production Usage — Where Spring Itself Uses AOP

- `@Transactional` (covered fully in the Spring Data JPA section) is IMPLEMENTED using AOP under the hood — Spring wraps your annotated method with advice that begins a transaction BEFORE it runs, and commits/rolls back AFTER it completes, WITHOUT you writing any of that transaction-management boilerplate yourself.
- `@Cacheable`, `@Async`, and Spring Security's method-level authorization checks (`@PreAuthorize`) are ALL implemented via the exact same AOP mechanism.

## 7. Edge Cases

| Case                                                                                           | Result                                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A method calls ANOTHER method on the SAME class internally ( <code>this.otherMethod()</code> ) | AOP advice does NOT apply to this INTERNAL, same-class call — Spring AOP only intercepts calls made THROUGH the Spring-managed proxy object, from OUTSIDE the class; a well-known, frequently-tested limitation |
| Overlapping pointcuts from MULTIPLE aspects matching the SAME method                           | Both apply, in an order that can be explicitly controlled via <code>@Order</code>                                                                                                                               |

## 8. Common Mistakes

- Expecting AOP advice to apply to INTERNAL, same-class method calls (`this.someMethod()`) — it only intercepts calls coming from OUTSIDE the class, through Spring's proxy mechanism.
- Writing overly broad pointcut expressions that unintentionally match FAR more methods than intended.

## 9. Frequently Asked Interview Questions

- 1 **What is AOP?** A programming paradigm letting you define cross-cutting concerns (logging, security, transactions) separately from core business logic, automatically applied across multiple methods/classes via Spring's proxy mechanism.
- 2 **What are "Aspect," "Advice," "Pointcut," and "Join Point"?** Aspect = the module containing cross-cutting logic; Advice = the actual action taken; Join Point = a point where advice CAN apply (a method call, in Spring); Pointcut = the expression defining WHICH join points the advice applies to.
- 3 **Name a real Spring feature built on top of AOP.** `@Transactional` — Spring wraps the annotated method with transaction begin/commit/rollback logic automatically, using the same AOP mechanism.

- 4 **What's a well-known limitation of Spring AOP regarding method calls?** It doesn't intercept INTERNAL, same-class method calls (`this.method()`) — advice only applies to calls made from OUTSIDE the class, through the Spring-managed proxy.
- 5 **What does `@Around` advice allow that other advice types don't?** Full control over the method call — it can run logic both before AND after, access/modify the return value, and even prevent the underlying method from executing at all.

## 10. Revision Notes (Entire Spring Core Section Consolidated)

### SPRING CORE – CHEAT SHEET

=====

IoC: hands CONTROL of object creation/wiring to the Spring CONTAINER (not your own code)

SOLVES: tight coupling, poor testability, rigid environment-specific configuration

DI: the SPECIFIC technique (constructor/setter/field injection) implementing IoC

CONSTRUCTOR injection is the STRONGLY preferred default (final fields, guaranteed complete init, trivially testable WITHOUT Spring at all)

BEAN: any object created/managed by the Spring IoC Container

BEAN LIFECYCLE: instantiate -> inject deps -> `@PostConstruct` -> [IN USE] -> `@PreDestroy` -> destroyed

BEAN SCOPES: singleton (DEFAULT, ONE shared instance - MUST be stateless!) vs prototype

(new instance EVERY request) vs request/session (web-scoped)

JAVA CONFIG: `@Configuration` + `@Bean` methods - explicit, type-safe, needed for 3rd-party classes

ANNOTATION CONFIG: `@Component`/`@Service`/`@Repository`/`@Controller`/`@RestController` on YOUR OWN classes

(all are specialized `@Component` - mainly semantic, `@Repository` also adds exception translation)

COMPONENT SCAN: auto-discovers `@Component` classes in the main class's package + SUB-packages ONLY

GOTCHA: a class OUTSIDE that tree is silently NEVER registered

AUTOWIRING: matches by TYPE by default

AMBIGUOUS (multiple matches) -> `NoUniqueBeanDefinitionException`, fix with `@Qualifier`/`@Primary`

NONE FOUND -> `NoSuchBeanDefinitionException`

AOP: cross-cutting concerns (logging, `@Transactional`, `@Cacheable`) applied automatically via

proxies, WITHOUT modifying the target class

Aspect/Advice/Pointcut/Join Point; `@Before`/`@After`/`@AfterReturning`/`@AfterThrowing`/`@Around`

GOTCHA: does NOT intercept internal `this.method()` calls, only external calls through the proxy

## 11. Homework (Covers Entire Spring Core Section)

- 1 Build a small `PaymentProcessor` interface with two implementations, use `@Qualifier` to explicitly wire one into a `@Service`, then switch to `@Primary` and observe the same result achieved differently.
- 2 Deliberately create two beans implementing the same interface WITHOUT any disambiguation, observe the exact `NoUniqueBeanDefinitionException` startup error, then fix it.
- 3 Write a simple `@Aspect` class that logs before and after every method call in a service package, and verify (via console output) it does NOT trigger for an internal `this.method()` call within the same class.
- 4 Create a `@Component` class in a package OUTSIDE your main application's package tree, observe that autowiring it elsewhere fails, then fix it via `@ComponentScan(basePackages = ...)`.

## 12. Mini Project — A Fully Wired Notification System (Demonstrating Everything)

```
package com.jobsradar.springcore;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Qualifier;
import org.springframework.context.annotation.Primary;
import org.springframework.stereotype.Component;
import org.springframework.stereotype.Service;
import java.util.List;

interface NotificationService {
    void notify(String message);
}

@Component
@Primary
class EmailNotificationService implements NotificationService {
    public void notify(String message) { System.out.println("EMAIL: " + message); }
}

@Component
class SmsNotificationService implements NotificationService {
    public void notify(String message) { System.out.println("SMS: " + message); }
}

@Service
class NotificationDispatcher {
    private final List<NotificationService> allServices; // autowires ALL implementations
    private final NotificationService primaryService; // autowires the @Primary one specifically

    @Autowired
    NotificationDispatcher(List<NotificationService> allServices,
        @Qualifier("emailNotificationService") NotificationService primaryService) {
        this.allServices = allServices;
        this.primaryService = primaryService;
    }

    public void broadcastToAll(String message) {
        allServices.forEach(s -> s.notify(message));
    }

    public void sendViaPrimary(String message) {
        primaryService.notify(message);
    }
}
```

---

## FINAL SUMMARY — SPRING CORE COMPLETE

This concludes the **Spring Core** section — 10 topics:

- 1 **IoC** — the tight-coupling/testability problem it solves, with a full before/after code comparison.
- 2 **Dependency Injection** — constructor vs setter vs field injection, why constructor wins.
- 3 **Beans** — what they are, the two ways to declare them.
- 4 **Bean Lifecycle** — `@PostConstruct`/`@PreDestroy`, the full phase diagram.

- 5 **Bean Scopes** — singleton (default, must be stateless!) vs prototype vs request/session.
- 6 **Java Configuration** — `@Configuration/@Bean`, replacing old XML config.
- 7 **Annotation Configuration** — `@Component/@Service/@Repository/@Controller/@RestController`.
- 8 **Component Scan** — auto-discovery, and the critical "outside the package tree" gotcha.
- 9 **Autowiring** — by-type matching, `@Qualifier/@Primary` for ambiguity.
- 10 **AOP Basics** — cross-cutting concerns, the terminology, and how `@Transactional` itself is built on this.

**You now understand not just Spring's syntax, but WHY the entire IoC/DI model exists — this is the direct, essential foundation for Spring Boot, Spring Data JPA, and Spring Security, all covered next in this course.**

---

*(END OF SPRING CORE MASTER NOTES — ready to proceed to Spring Boot whenever you say "Next Topic.")*